

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

Enunciat de la pràctica de laboratori

Lab 3:
Display 7 segments

L3. Display 7 segments

L3(A) Multiplexat en el temps del display

L3(B) Interrupcions generades per botons

1 Objectius

L'objectiu d'aquesta pràctica és controlar un grup de 4 displays de 7-segments de manera que el microcontrolador pugui presentar informació a través de les seves sortides.

Durant la primera part de la pràctica L3(A), farem servir botons d'entrada i consultarem el seu estat pel mètode d'enquesta (polling). Segons l'estat dels botons anirem manipulant els 4 displays per pintar un número mitjançant la tècnica de multiplexat en el temps.

Durant la segona part de la pràctica L3(B), l'objectiu serà la programació de les rutines de servei a les interrupcions (RSI). Això ens permetrà no haver de consultar constantment l'estat dels botons d'entrada. Amb aquest mètode es podrà generar una interrupció al apretar (o al deixar anar) el botó de tal manera que s'executin codis específics a dintre del microcontrolador. Per fer-ho, s'haurà de comprendre la configuració dels diferents paràmetres que intervenen en la gestió de les interrupcions (habilitació, prioritats, flancs d'activació, etc.).

En acabar la pràctica l'alumne serà capaç de:

- manegar informació tècnica donada pel fabricant de la placa de desenvolupament EasyPIC v7 que utilitzem a les pràctiques.
- cercar la informació necessària dins dels manuals de referència.
- aprofundir el coneixement dels PORTS d'E/S.
- controlar l'activació d'un display de 7-segments per fer-hi un dibuix desitjat.
- entendre el concepte de multiplexat en temps, per aconseguir mostrar un nombre de diversos dígits al conjunt de 7-segments.
- solucionar problemes d'implementació com el *ghosting* que no són visibles durant la simulació
- entendre la diferència entre enquesta o interrupció per la gestió dels botons
- utilitzar els botons de forma que executin codi diferent quan estan apretats, quan no estan apretats, quan es produeix un flanc de pujada o quan es produeix un flanc de baixada
- entendre com configurar interrupcions en el PIC18F45K22
- entendre com implementar les rutines de servei a la interrupció
- entendre el concepte de flag de la interrupció
- entendre la utilitat d'activar o desactivar interrupcions

2 Introducció

La placa de desenvolupament EasyPIC v7 (Fig.1) disposa de 4 displays de 7 segments que ens permetran mostrar informació de manera senzilla, emprant els ports d'E/S.

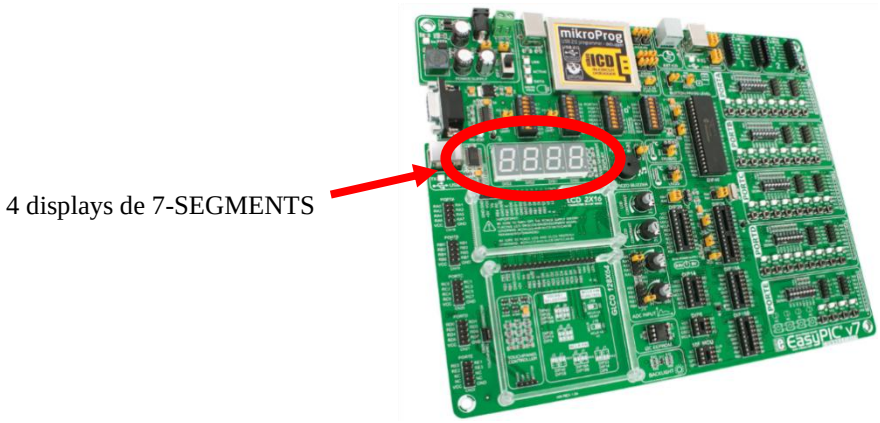


Fig. 1. Detall dels displays de 7 segments a la plataforma de desenvolupament EasyPIC v7.

Aquests 4 displays de 7 segments estan connectats amb una configuració anomenada "Càtode Comú", que vol dir que els 8 leds del display 7-segments (7 per crear la figura i un pel punt decimal) comparteixen una connexió a massa, i per tant són actius a "1". La figura 2 mostra la connexió dels quatre displays extreta del manual de la placa. Podeu recuperar la pràctica anterior on s'explica breument la funcionalitat de les diferents parts que formen aquest circuit.

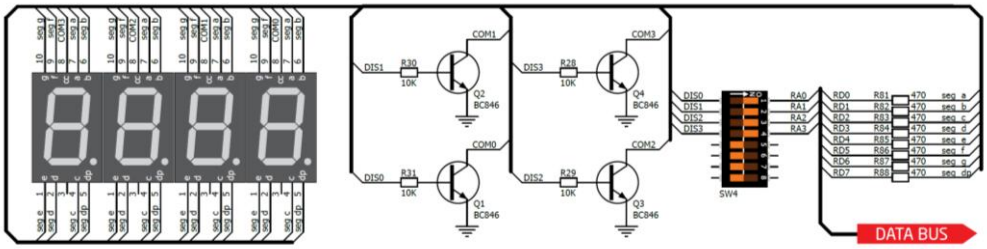


Fig. 2. Connexió a la placa dels 4 7-segments.

Observant les connexions a la placa podem concloure que:

- L'activació dels leds del display depèn del PORTD.
- **El PORTD està connectat als quatre displays simultàniament.**
- La selecció del display actiu es fa mitjançant el càtode comú, amb uns senyals anomenats DIS0, DIS1, DIS2 i DIS3.
- Els senyals DISx es poden connectar al PORTA mitjançant el switch SW4.

Finalment, tal i com es va explicar a la pràctica anterior, veiem que el circuit de selecció del display actiu, amb càtode comú, el regeix un transistor tipus NPN i que s'activarà per "1", és a dir, quan posem un "1" a la senyal DISx s'activa el corresponent display.

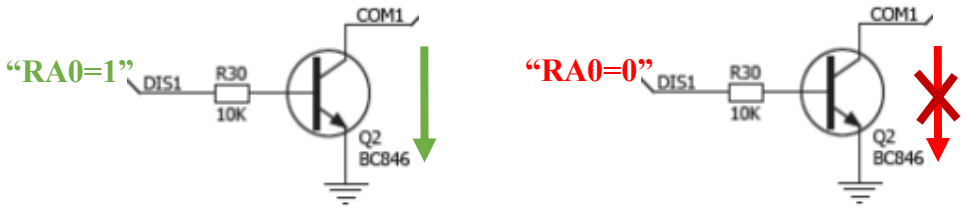


Fig. 3. Circuit d'activació del display de 7-segments, quan al pin RA0 hi ha un "1" el transistor condueix i el display s'encén, quan al RA0 hi ha un 0 el transistor no condueix i el display resta apagat.

Per pintar en el display s'ha de seleccionar un únic display DIS0, DIS1, DIS2 o DIS3 amb el PORTA i tot seguit enviar pel PORTD el valor a pintar. A continuació s'espera una mica, es selecciona el següent display i se li envia el nou número i així successivament. Aquesta tècnica s'anomena "multiplexat en el temps". Si s'espera molt temps entre un dígit i el següent es veuran displays que s'encenen i s'apaguen donant una resultat poc atractiu. Si s'espera poc temps es barrejaran els valors pintats al conjunt. Teniu més informació disponible a Atenea a l'*Annex: info7Seg.pdf*. Llegiu-la amb atenció ja que serà una part molt important de la pràctica. En particular, mireu el punt 3 del document on es fa referència al problema del ghosting, que és un problema d'implementació que haureu de solucionar al laboratori.

A continuació s'inclouen una sèrie de passos per simular els 4 displays de 7-segments a Proteus:

2.1- Incloure l'arxiu `config.h` al projecte de Proteus. Aquest arxiu està disponible a Atenea.

2.2- Configurar la simulació del PIC18F45K22 a Proteus perquè funcioni a 10MHz de tal manera que la simulació i la placa EasyPicv7 funcionin a la mateixa velocitat de rellotge. Per fer-ho heu d'anar a la pestanya de l'esquemàtic i fer doble click sobre el microcontrolador. S'obrirà una finestra per editar el component i s'haurà de modificar el valor de Processor Clock Frequency a 10MHz.

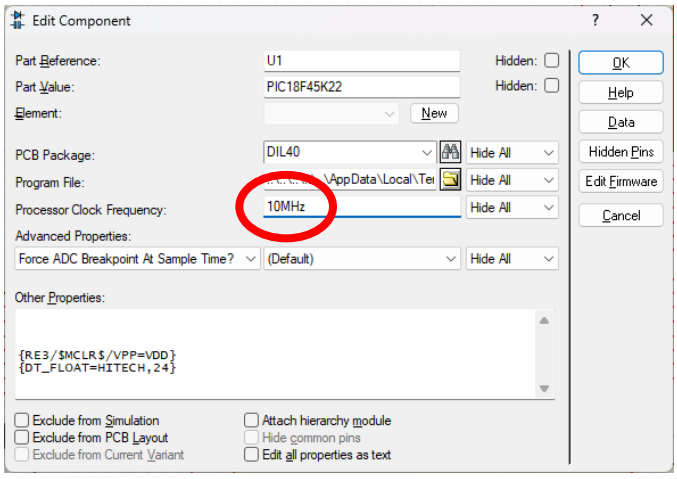


Fig. 4. Captura de pantalla pel canvi de la velocitat del clock.

2.3-Disseny de l'esquema elèctric sobre Proteus. Farem el disseny amb Proteus amb el microcontrolador PIC18F45K22 i hi inclourem el bloc de 4 displays de 7-segments "7SEG-MPX4-CC" que funciona en càtode comú, tal com necessitem. Per gestionar la selecció dels displays, canviarem el circuit real amb els transistors NPN per portes NOT. A nivell lògic funcionen igual (al posar un "1" a la porta, surt un "0" del negador que fa que el display corresponent estigui connectat a "0" ó terra i per tant es tanca el circuit i es pot il·luminar el display) i estalviem que Proteus hagi de fer la simulació d'un circuit analògic amb els transistors, que ralentitzaria molt la simulació.

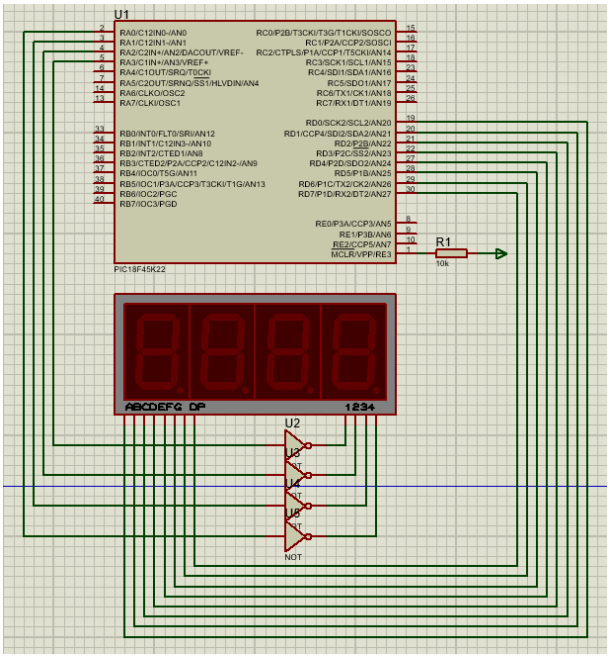


Fig. 6. Model per l'esquema de Proteus.

2.4-Podeu implementar l'espera entre un dígit i el següent amb `__delay_us(x)` on `x` és el temps d'espera en microsegons o `__delay_ms(x)` per fer esperar `x` milisegons (vigileu que té **2** guions baixos al davant). Per fer servir aquests delays necessiteu afegir `#define _XTAL_FREQ 10000000` al principi del vostre codi per indicar a la funció que la freqüència del clock és de 10MHz. Consulteu els arxius sobre la Freqüència Fosc del PIC *infoFosc.pdf* i *infoGraficIncludesXC8_PIC18F45K22.pdf* que trobareu a Atenea per a més informació.

3 Treball Previ Multiplexat en el temps L3 (A)

Fer un programa en llenguatge C amb les següents funcionalitats:

3.1- Llegir l'estat del pin RC0 per enquesta. Cada cop que es detecti un flanc de baixada en aquest pin generarà un número aleatori entre 0 i 9999 que es pintarà al display.

3.2- Llegir l'estat del pin RC1 per enquesta. Cada cop que es detecti un flanc de pujada en aquest pin es suma +1 al número que apareix als displays. Si per exemple el numero aleatori era el 3479, llavors apretar el RC1 farà que aparegui el número 3480.

3.3- Llegir l'estat del pin RC2 per enquesta. Cada cop que es detecti un flanc de pujada en aquest pin es resta -1 al número que apareix als displays.

3.4- Llegir l'estat del pin RC7 per enquesta. Cada cop que es detecti un flanc de baixada en aquest pin pinti el valor 0000 als displays.

3.5- Llegir l'estat del pin RC6 per enquesta. Aquest pin servirà per encendre/apagar els displays. Al principi els displays estaran activats, un flanc de baixada al pin RC6 farà que s'apaguin. Posteriorment, un altre flanc de baixada al pin RC6 farà que s'encenguin els displays. La resta de funcionalitats descrites anteriorment han d'estar actives encara que estiguin apagats tots els displays.

Per generar números aleatoris podeu fer servir `srand()` i `rand()`, i necessitareu incloure la llibreria `#include <stdlib.h>`. Trobareu informació sobre aquestes i altres rutines i com utilitzar-les al manual de referència del compilador: "*MPLAB XC8 C Compiler User's Guide*", secció "*Appendix A. Library Functions*". Podreu trobar l'explicació sobre `rand`, `srand`, així com les rutines de delay `__delay_ms` i `__delay_us`. Trobareu aquest document a Atenea amb el nom Compilador XC8 al link:

`https://atenea.upc.edu/pluginfile.php/7183837/mod_resource/content/3/xc8.pdf`

Podeu passar pels laboratoris de l'assignatura en l'horari d'accés lliure per comprovar el funcionament del vostre codi i solucionar problemes. D'aquesta manera podreu garantir que el que funciona al simulador també funciona a la placa ja que no sempre és així donat que Proteus no pot simular totes les configuracions, ni el soroll o pertorbacions, etc. També podeu treballar en algun mètode de generació de números aleatoris que no es repeteixi cada cop que es faci reset.

4 Rúbrica treball Previ Multiplexat en el temps L3 (A)

	Iniciat (0-2.5 punts)	En desenvolupament (2.5-5.0 punts)	Aconseguit (5.0-7.5 punts)	Exemplar (7.5-10 punts)
Botons (2 punt):	Hw mal dissenyat i funcionament erroni	Hw mal dissenyat o funcionament erroni	Funcionament correcte però mal ús de recursos	Funciona perfectament
Detecció flancs (3 punts):	La detecció és inexistent	Alguna part de la detecció és errònia	La detecció o no és estructurada, o no és modular, o no està comentada o té algun error	La detecció és estructurada, forma part d'una llibreria, és modular, està comentada i no té cap error
Números aleatoris (3 punts):	No hi ha cap generació de números aleatoris	La seqüència de números aleatoris és sempre la mateixa	La seqüència de números aleatoris és repeteix en algunes condicions	A cada execució apareix una sèrie diferent de números aleatoris
Display 7-segments (2 punts):	Hw mal dissenyat	Visualització incorrecta als displays	Codi poc eficient amb especial èmfasis en com es mapegen els números als displays i el multiplexat en el temps	Funciona perfectament

5 Treball Previ Interrupcions per botons L3 (B)

El programa a desenvolupar en aquesta segona part de la pràctica consisteix en pintar al display de 7-segments un número mitjançant els botons associats a les interrupcions INT0, INT1 i INT2, per tant **haureu d'escriure el codi d'atenció a la interrupció corresponent (no és correcte fer servir consulta o polling per les interrupcions)**. El codi a implementar ha de complir els següents requeriments:

5.1. Al principi del programa hi apareixerà el número "0000" als displays. La INT0 estarà activa, mentre que les INT1 i INT2 estaran desactivades.

5.2. Al apretar la primera vegada el botó de la INT0, es detectarà el flanc de pujada i es pintarà un número aleatori entre 0 i 9999. En aquest moment s'activaran les INT1 i INT2.

5.3 Al apretar la primera vegada el botó de la INT1, es detectarà el flanc de baixada i es seleccionarà el primer dígit del display començant per la dreta. Per indicar que està seleccionat aquest dígit, s'activarà el punt de la part inferior dreta de cada display, també anomenat DP (recupereu la Fig. 7 de l'enunciat de la pràctica anterior on s'observa que el led DP està connectat al pin RD7).

5.4 Al apretar una segona vegada el botó de la INT1, es detectarà el flanc de baixada i es seleccionarà el segon dígit començant per la dreta.

5.5. Es repetirà la detecció del flanc de baixada de INT1 fins arribar al 4rt dígit. Si es torna a detectar un flanc de baixada addicional, llavors es torna a seleccionar el primer dígit.

5.6. Si en qualsevol moment es detecta una interrupció per flanc de baixada associada al botó de la INT2, llavors s'incrementa en una unitat el valor del dígit seleccionat. Si el valor d'aquest dígit és més gran de 9 es posarà un 0. Al contrari que en el cas anterior a la pràctica L3(A), ara s'incrementa només el dígit en comptes del número complet.

5.7. Si en qualsevol moment es torna a apretar la INT0, es posarà el número "0000" i es desactivaran les INT1 i INT2 fins que es torni a apretar INT0 on tornaríem al punt 5.2.

5.8- Les interrupcions són un punt crític a la programació de microcontroladors degut a que la seva natura asíncrona fa que sigui difícil debugar, tracejar i testejar el codi. Per tant, és necessari ser meticulós en la fase de desenvolupament per evitar errors crítics. En el nostre PIC farem algunes proves per veure que hem programat adequadament. En primer lloc posarem un software breakpoint a la rutina de servei a la interrupció que gestiona INT0 i mirarem que l'execució del codi s'atura correctament i s'executa el codi corresponent fent servir el botó F10 per avançar línia a línia mentre debuguem. Indiqueu al codi, al costat del codi d'atenció a la interrupció un comentari amb el millor i el pitjor temps d'execució del vostre codi de la interrupció. Podeu posar dos breakpoints diferents i amb el botó de continuar la simulació podreu avançar d'un a l'altre. A la part baixa del programa veureu el temps transcorregut

5.9- Posarem un hardware breakpoint al pin INT0 que s'activi amb el flanc de pujada. Podeu veure el vídeo tutorial Breakpoints i Hardware Breakpoints en la web de Proteus `https://www.labcenter.com/tutorials/` o a youtube amb l'usuari Labcenter Electronics Ltd `https://www.youtube.com/watch?v=qFkcELwgBuM`.

5.10- Estresseu el sistema per a forçar l'aparició de bugs no detectats. Aquesta tècnica consisteix en provocar moltes més interrupcions per segon que les que s'esperarien en funcionament normal. Per a tal fi, substituïu els polsadors per un generador de funcions que generi interrupcions a una freqüència molt elevada o un generador de polsos que trobareu al menú Generators. Podeu *seleccionar Analogue Types = Pulse, Pulse (High) Voltage = 5V,*

i anar canviant el valor de *Frequency*. Comproveu fins a quina freqüència màxima pot arribar el generador abans de que el vostre codi deixi de funcionar correctament. Indiqueu el valor en un altre comentari al vostre codi.

Podeu passar pels laboratoris de l'assignatura en l'horari d'accés lliure per comprovar el funcionament del vostre codi i solucionar problemes. D'aquesta manera podreu garantir que el que funciona al simulador també funciona a la placa ja que no sempre és així donat que Proteus no pot simular totes les configuracions, ni el soroll o pertorbacions, etc. També podeu treballar en el test d'estrès de les interrupcions (indiqueu amb comentaris al codi les proves que heu fet al laboratori i descriviu el passos realitzats).

6 Rúbrica treball Previ Interrupcions per botons L3 (B)

	Iniciat (0-2.5 punts)	En desenvolupament (2.5-5.0 punts)	Aconseguit (5.0-7.5 punts)	Exemplar (7.5-10 punts)
Botò INT0 (1.5 punts):	Hw mal dissenyat i funcionament erroni	Hw mal dissenyat o funcionament erroni (flanc de pujada)	Funcionament correcte però mal ús de recursos o mala configuració	Funciona perfectament i les interrupcions associades estan ben gestionades
Botò INT1 (1.5 punts):	Hw mal dissenyat i funcionament erroni	Hw mal dissenyat o funcionament erroni (flanc de pujada)	Funcionament correcte però mal ús de recursos o mala configuració	Funciona perfectament i les interrupcions associades estan ben gestionades
Botò INT2 (1.5 punts):	Hw mal dissenyat i funcionament erroni	Hw mal dissenyat o funcionament erroni (flanc de baixada)	Funcionament correcte però mal ús de recursos o mala configuració	Funciona perfectament i les interrupcions associades estan ben gestionades
Display 7-segments (3 punts):	Hw mal dissenyat	Visualització incorrecta als displays	Codi poc eficient	Funciona perfectament
Temps d'execució i freqüència interrupcions (2.5 punts):	No hi ha resposta	La resposta no és coherent	La resposta no està raonada	La resposta és raonada, justificada i coherent